

Full Stack Development with MERN

Project Documentation

1. Introduction

• **Project Title:** Online Complaint Registration and Management System

• **Team Members:**

N. Sri Gowri Naga Hymavathi – Full Stack Developer (Frontend, Backend, Database, Testing).

2. Project Overview

Purpose

The purpose of this project is to provide a digital platform for users to register complaints, track their status, and communicate with authorities efficiently. It replaces manual complaint systems with a faster and more transparent solution.

Features

- User Registration and Login
- Complaint Submission with Image Upload
- Complaint Status Tracking
- Role-based Access (User, Agent, Admin)
- Admin Complaint Assignment
- Agent Status Updates
- Dashboard for all users

3. Architecture

Frontend:

- Built using React.js with component-based architecture
- 13 components organized into 4 folders — admin, agent, common, user
- Uses Axios library for REST API communication with the backend
- UI designed with Bootstrap for responsive layout and Material UI for polished components

- React Router DOM for client-side navigation between pages
- Role-based routing — users go to /home, agents to /agent, admins to /admin

Backend:

- Built using Node.js and Express.js
- Implements MVC (Model-View-Controller) architecture
- View layer — 4 route files defining REST API endpoints
- Controller layer — 4 controller files handling business logic
- Model layer — 3 Mongoose schema files for data modeling
- Uses JWT (JSON Web Token) for authentication and authorization
- Multer middleware for handling image/file uploads
- bcryptjs for password hashing and security
- CORS enabled for frontend-backend communication

Database:

- MongoDB NoSQL database running locally on port 27017
- Database name: complaintdb
- Uses Mongoose ODM for schema definition and database operations
- 3 collections: users, complaints, messages

4. Setup Instructions

Prerequisites:

- Node.js v18 or above
- MongoDB / MongoDB Compass
- npm (Node Package Manager)
- VS Code (recommended editor)

Installation:

Step 1 — Clone or download the project folder

Step 2 — Setup Backend:

```
cd backend  
npm install
```

Create a .env file with:

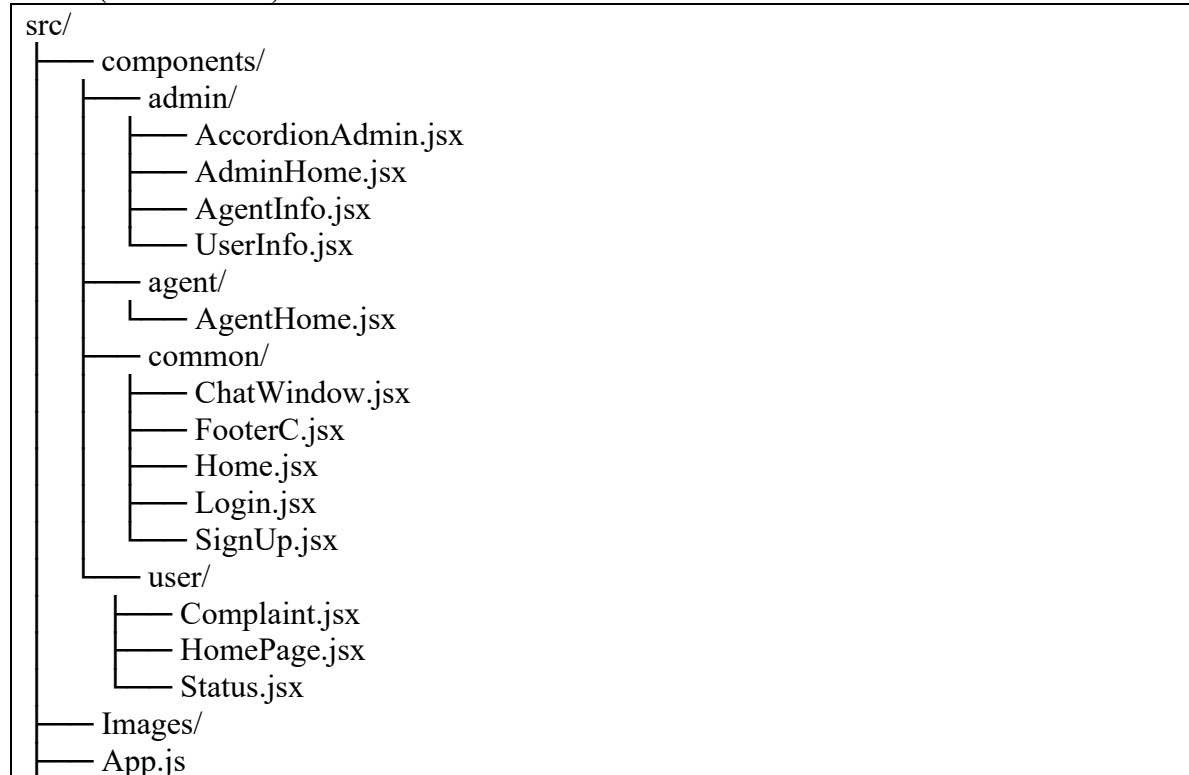
```
PORT=5000  
  
MONGO_URI=mongodb://localhost:27017/complaintdb  
  
JWT_SECRET=complaintsecret123
```

Step 3 — Setup Frontend:

```
cd frontend  
npm install
```

5. Folder Structure

Client (frontend/src/):



```
├── App.css
├── index.js
```

Server (backend/):

```
backend/
├── controllers/
│   ├── authController.js
│   ├── complaintController.js
│   ├── adminController.js
│   └── messageController.js
├── middleware/
│   └── authMiddleware.js
├── models/
│   ├── User.js
│   ├── Complaint.js
│   └── Message.js
├── routes/
│   ├── authRoutes.js
│   ├── complaintRoutes.js
│   ├── adminRoutes.js
│   └── messageRoutes.js
├── uploads/
├── .env
└── server.js
```

6. Running the Application

Start Backend:

```
cd backend
node server.js
```

Backend runs at: <http://localhost:5000>

Start Frontend:

```
cd frontend
npm start
```

Frontend runs at: <http://localhost:3000>

Open browser and go to <http://localhost:3000> to use the application.

7. API Documentation

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Register new user	No
POST	/api/auth/login	Login and get JWT token	No
POST	/api/complaints	Submit new complaint	Yes (User)
GET	/api/complaints/my	Get logged-in user's complaints	Yes (User)
GET	/api/complaints/:id	Get complaint by ID	Yes
PUT	/api/complaints/:id/status	Update complaint status	Yes (Agent)
GET	/api/complaints/assigned	Get agent's assigned complaints	Yes (Agent)
GET	/api/admin/dashboard	Get all complaints and stats	Yes (Admin)
PUT	/api/admin/assign	Assign complaint to agent	Yes (Admin)
GET	/api/admin/agents	Get all agents	Yes (Admin)
GET	/api/admin/users	Get all users	Yes (Admin)
DELETE	/api/admin/users/:id	Delete a user	Yes (Admin)
DELETE	/api/admin/agents/:id	Delete an agent	Yes (Admin)
GET	/api/messages/:complaintId	Get messages for a complaint	Yes
POST	/api/messages	Send a message	Yes

8. Authentication

The project uses JWT (JSON Web Token) based authentication:

- When a user registers or logs in, the server generates a JWT token signed with a secret key stored in the .env file
- The token contains the user's ID and role (user/agent/admin)
- The token is stored in the browser's localStorage
- Every protected API request sends the token in the Authorization header as Bearer <token>
- The authMiddleware.js verifies the token on every protected route
- Role-based middleware (adminOnly, agentOnly) restricts access based on the user's role
- Passwords are hashed using bcryptjs before storing in MongoDB — plain passwords are never saved

9. User Interface

The application has three separate dashboards:

User Interface:

- Home landing page with feature cards
- SignUp and Login forms
- User dashboard showing all filed complaints
- Complaint submission form with image upload
- Status tracking page with visual timeline
- Chat window for communicating with agent

Agent Interface:

- Agent dashboard showing assigned complaints list
- Complaint detail view with status update buttons
- Chat window for communicating with user

Admin Interface:

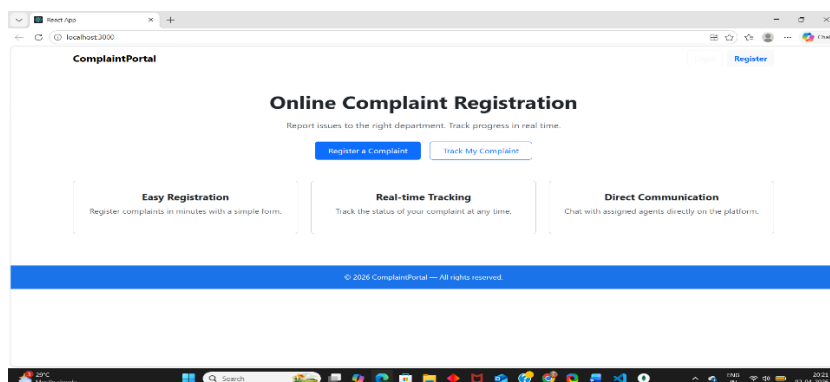
- Admin dashboard with stats cards (Total, Pending, In Progress, Resolved)
- All complaints list with assign agent option
- Agent management tab — view and remove agents

- User management tab — search, view and remove users

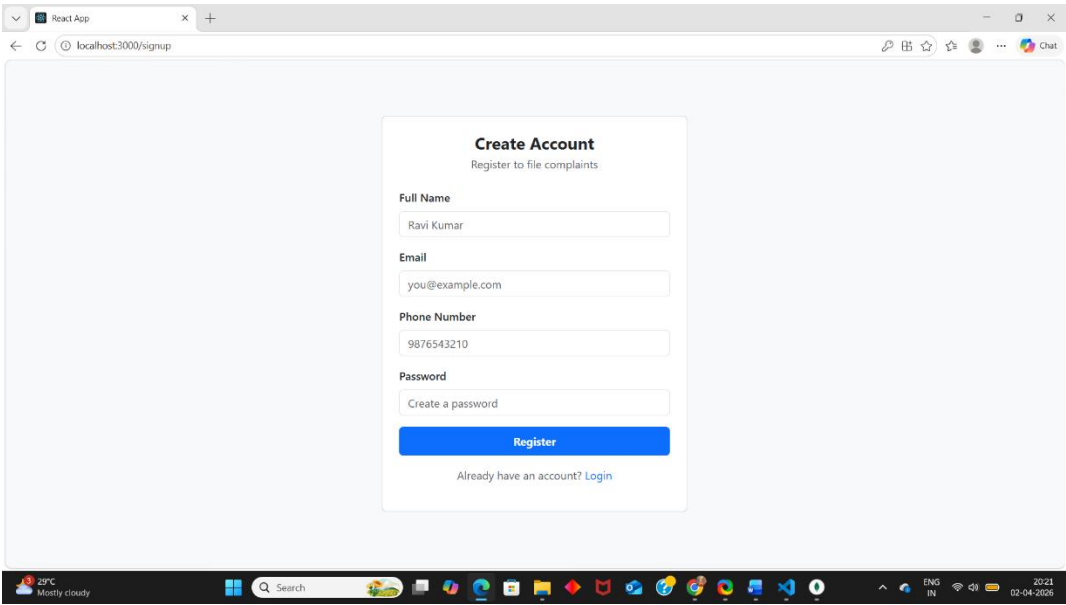
10. Testing

Test Type	Method	Result
User Registration	Filled form with valid data → registered successfully	Pass
User Login	Logged in with correct credentials → JWT token generated	Pass
Role-based Redirect	Admin login → redirected to /admin, Agent → /agent	Pass
Complaint Submission	Filled form with image → saved in MongoDB	Pass
Status Tracking	Clicked Track → showed correct status timeline	Pass
Admin Assignment	Admin assigned agent → complaint status changed to Assigned	Pass
Agent Status Update	Agent updated status → reflected on user dashboard	Pass
Chat Messaging	User sent message → agent received it	Pass
Invalid Login	Wrong password → error message shown	Pass
MongoDB Data	Verified all collections in MongoDB Compass	Pass

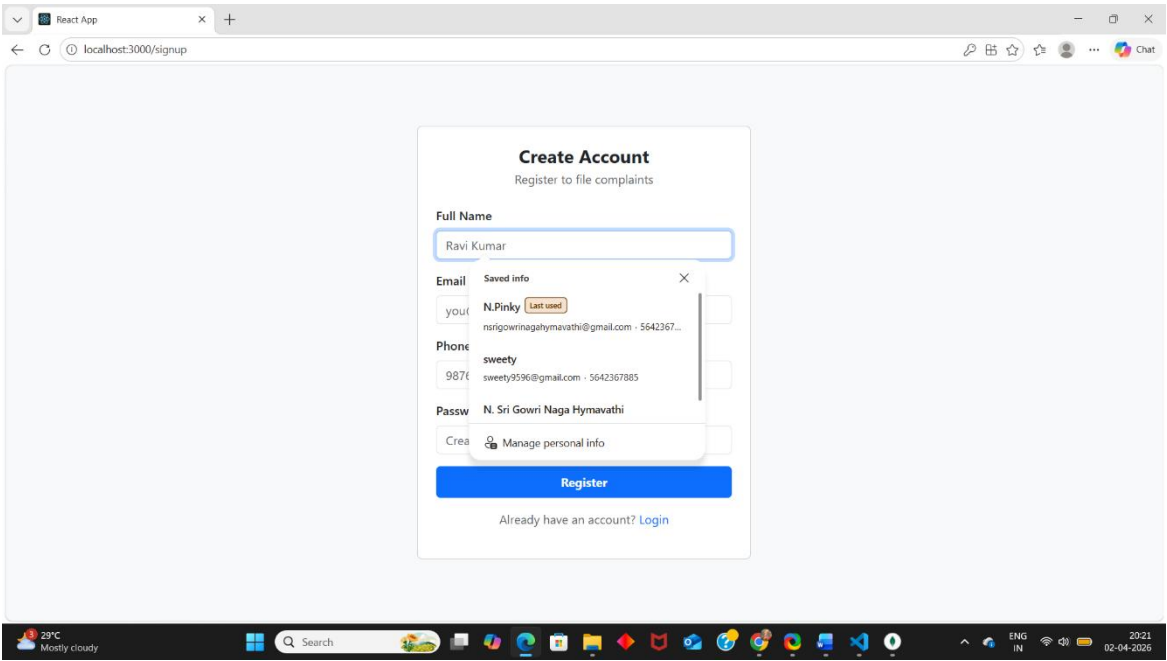
11. Screenshots or Demo



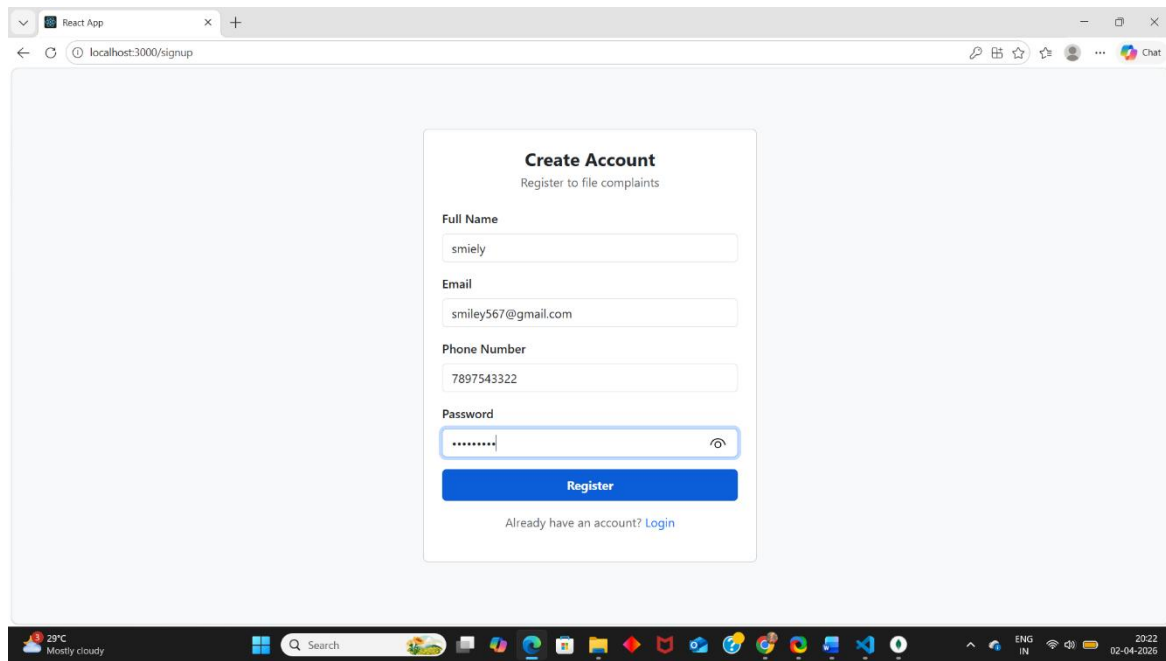
This is my initial page



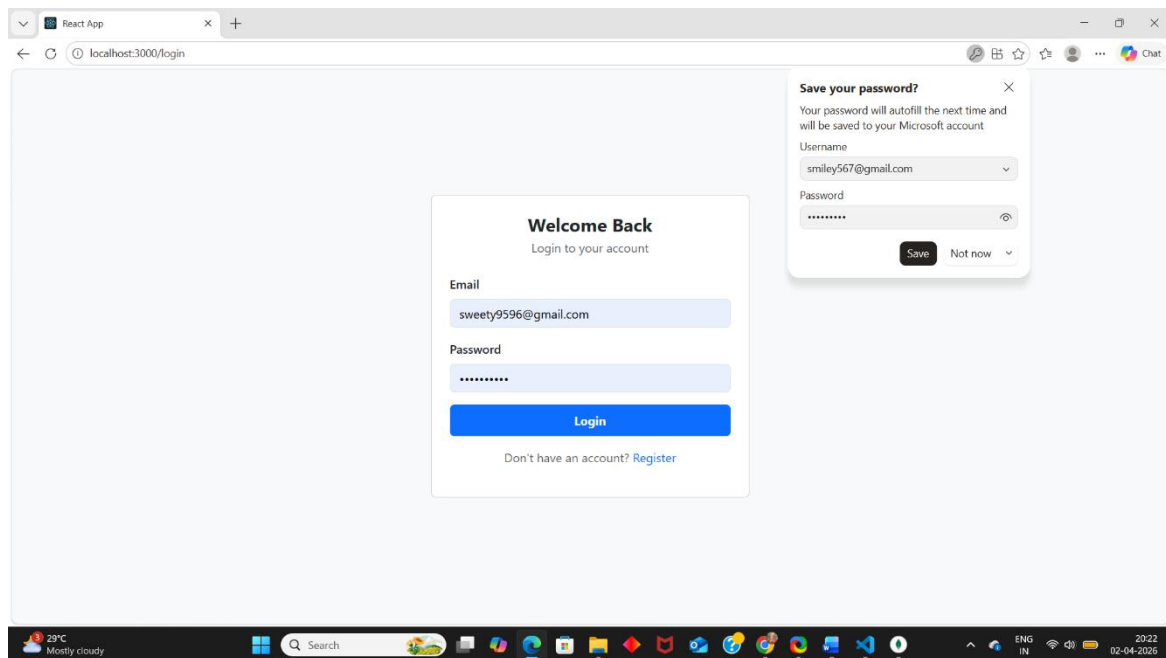
Here you can create an account or login

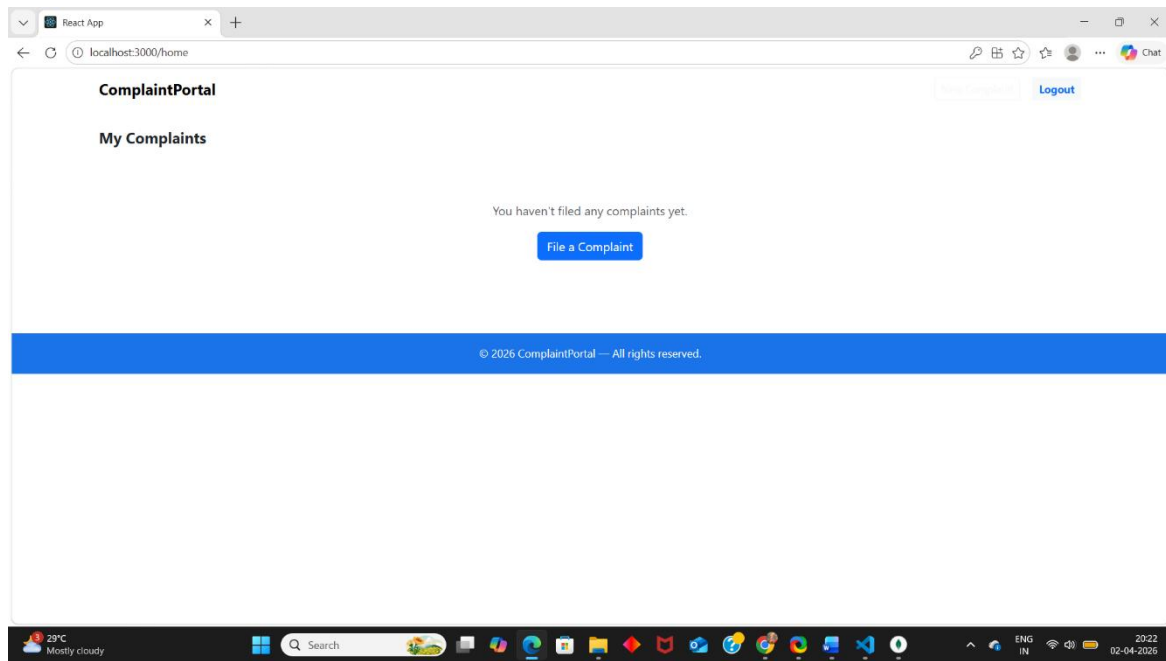


Here it is showing previous registrations

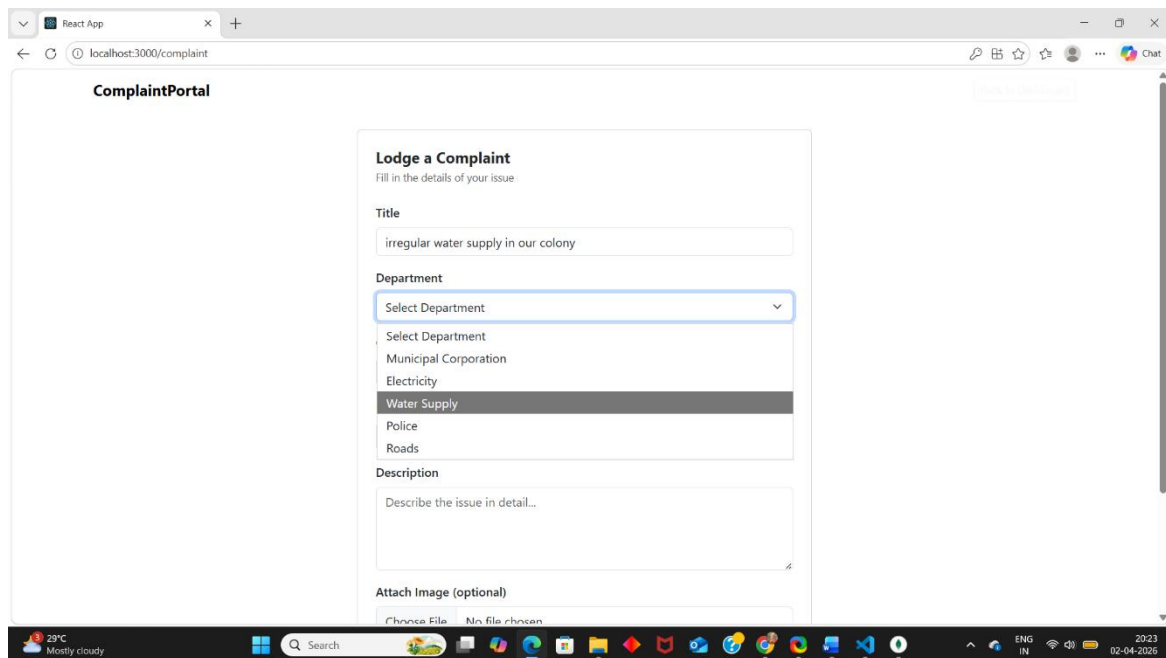


I have created new account





This is my home page



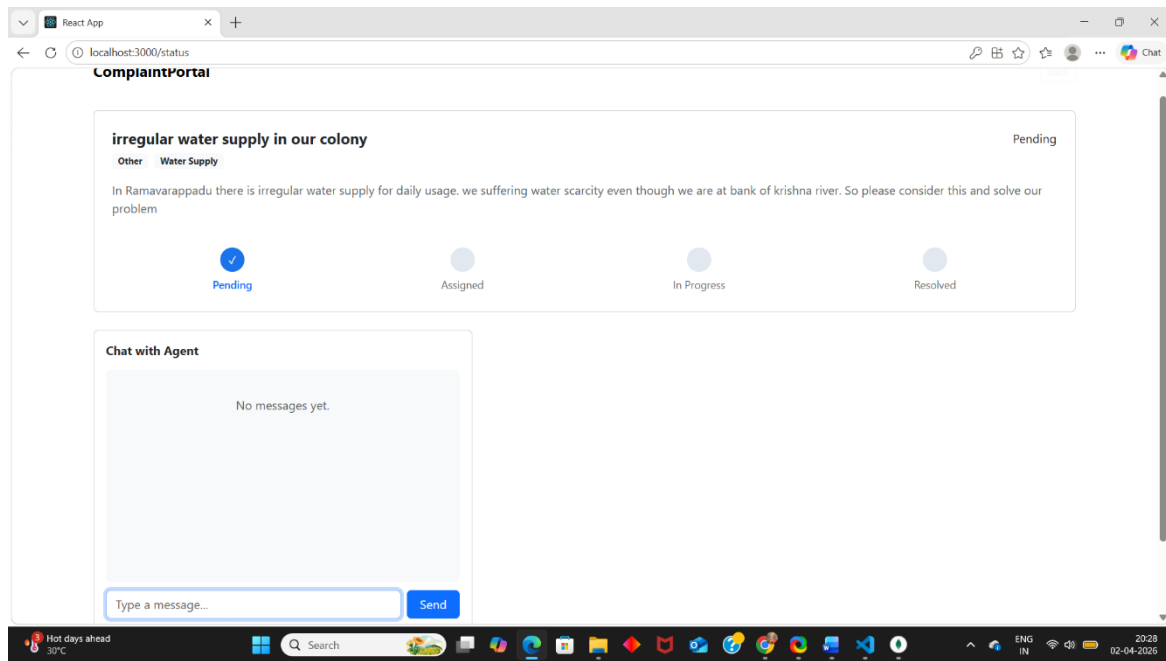
Here I am giving a complaint

A screenshot of a web browser showing the 'ComplaintPortal' form submission page. The browser's address bar shows 'localhost:3000/complaint'. The form contains the following fields: a text input with 'irregular water supply in our colony', a 'Department' dropdown menu set to 'Water Supply', a 'Category' dropdown menu set to 'Other', an 'Urgency Level' dropdown menu set to 'High', and a 'Description' text area with the text: 'In Ramavarappadu there is irregular water supply for daily usage. we suffering water scarcity even though we are at bank of krishna river. So please consider this and solve our problem'. Below the description is an 'Attach Image (optional)' section with a 'Choose File' button and a file name 'irreggular wtaer supply.jpg'. A blue 'Submit Complaint' button is at the bottom of the form. The footer of the page shows '© 2026 ComplaintPortal — All rights reserved.' and a Windows taskbar at the bottom with the date '02-04-2026' and time '20:27'.

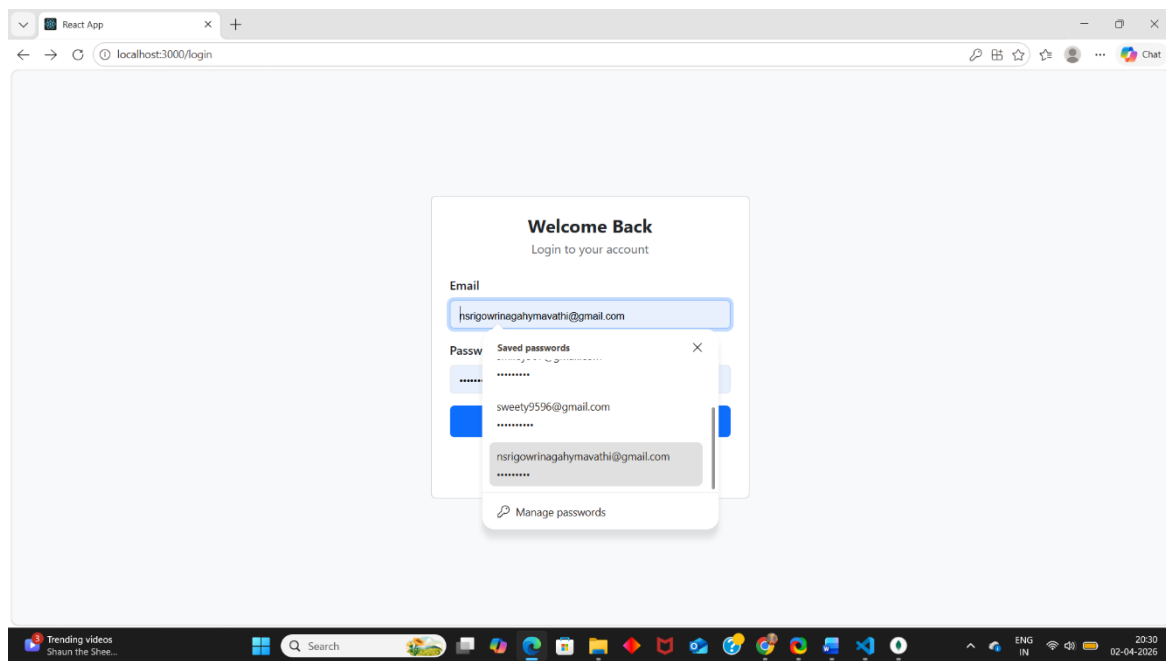
I completed giving complaint and clicked on submit

A screenshot of the 'ComplaintPortal' 'My Complaints' page. The browser's address bar shows 'localhost:3000/home'. The page has a header with 'ComplaintPortal' and a 'Logout' button. The main section is titled 'My Complaints' and displays a single complaint card. The card has the title 'irregular water supply in our colony', a status of 'Pending', and the same description as the previous screenshot. Below the description is a 'Track' button. The footer of the page shows '© 2026 ComplaintPortal — All rights reserved.' and a Windows taskbar at the bottom with the date '02-04-2026' and time '20:28'.

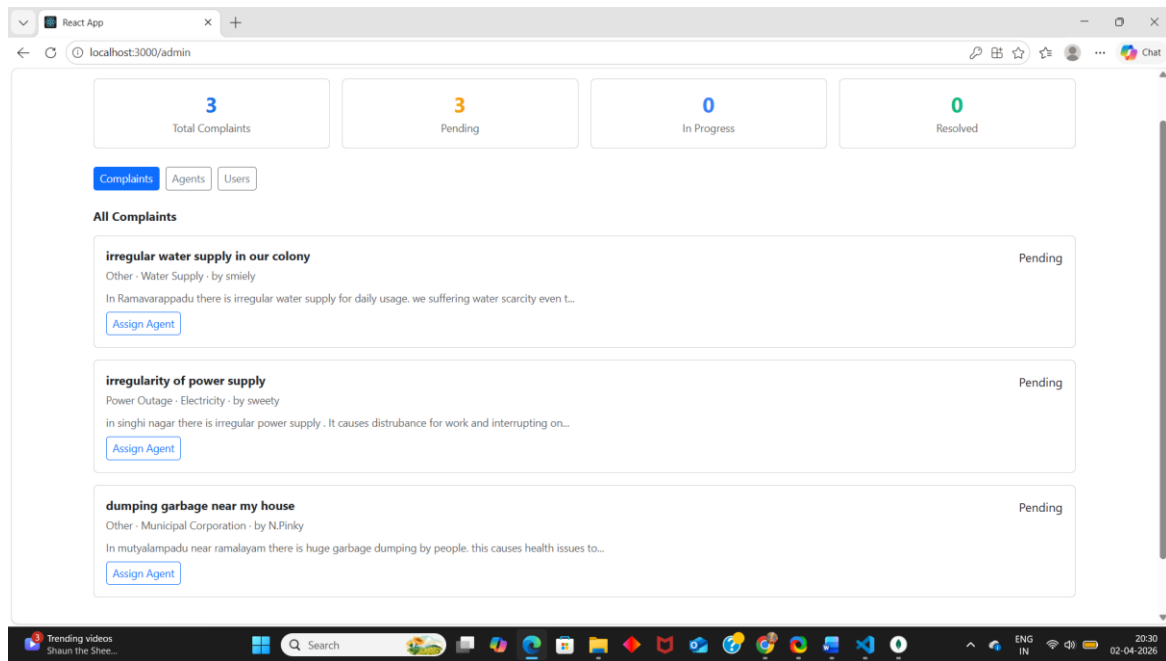
My complaint is updated here



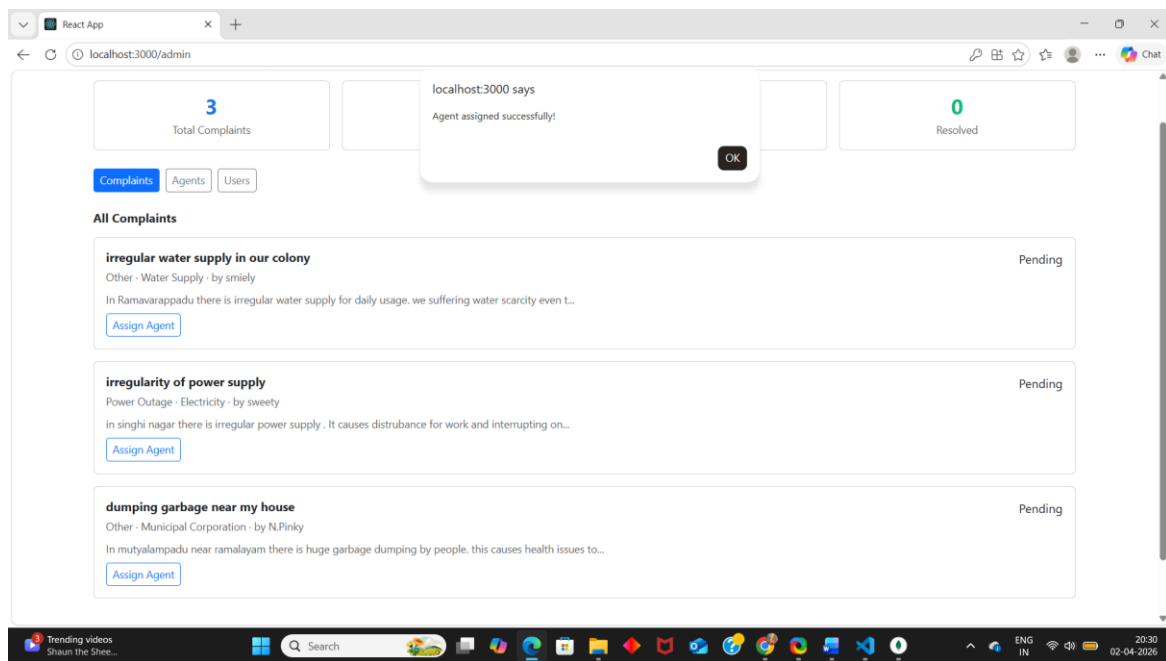
This is showing my progress in status bar



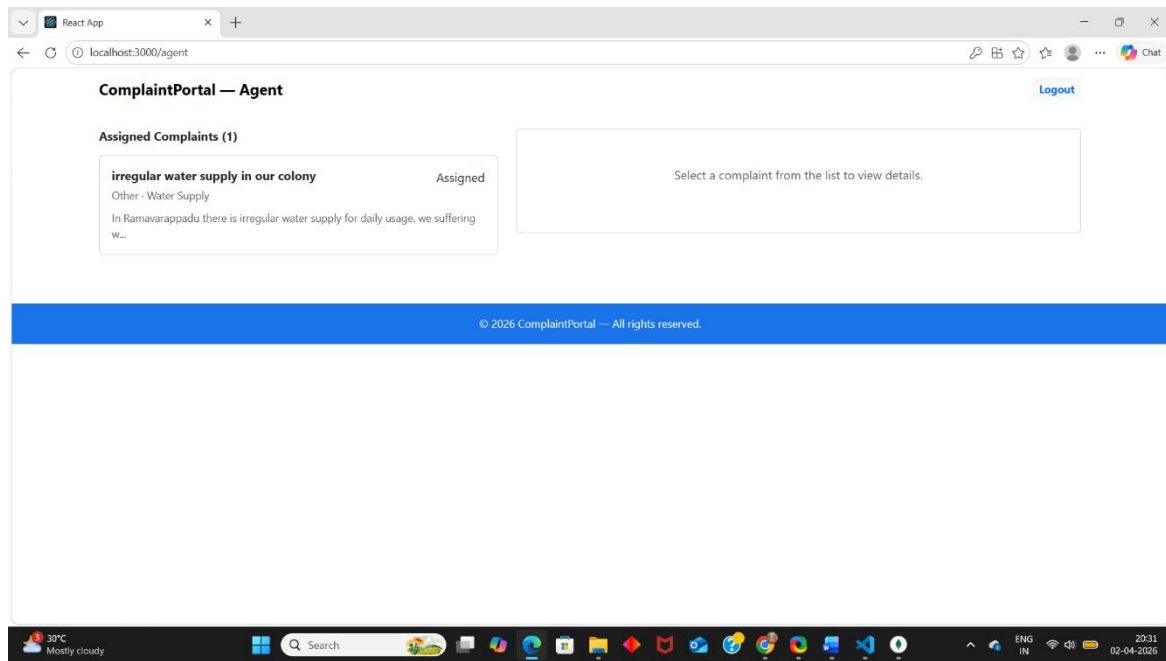
Now I am logging in into account that I have created before



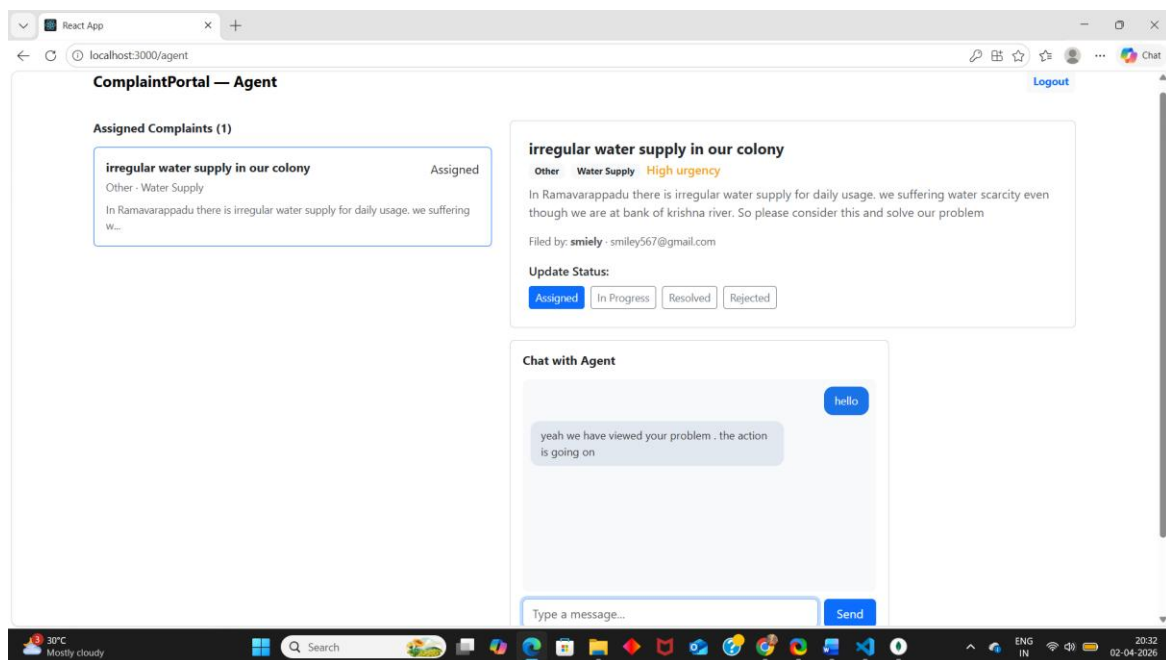
Here I am seeing all complaints by users in admin bar



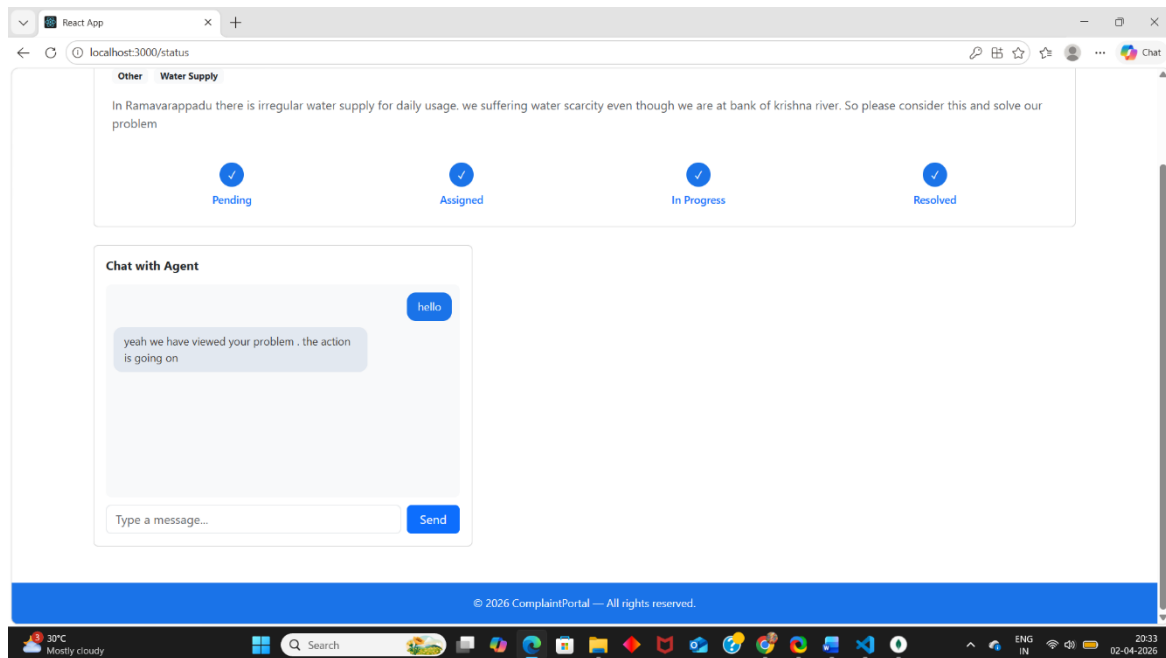
Here I assigned an agent to user to resolve problem



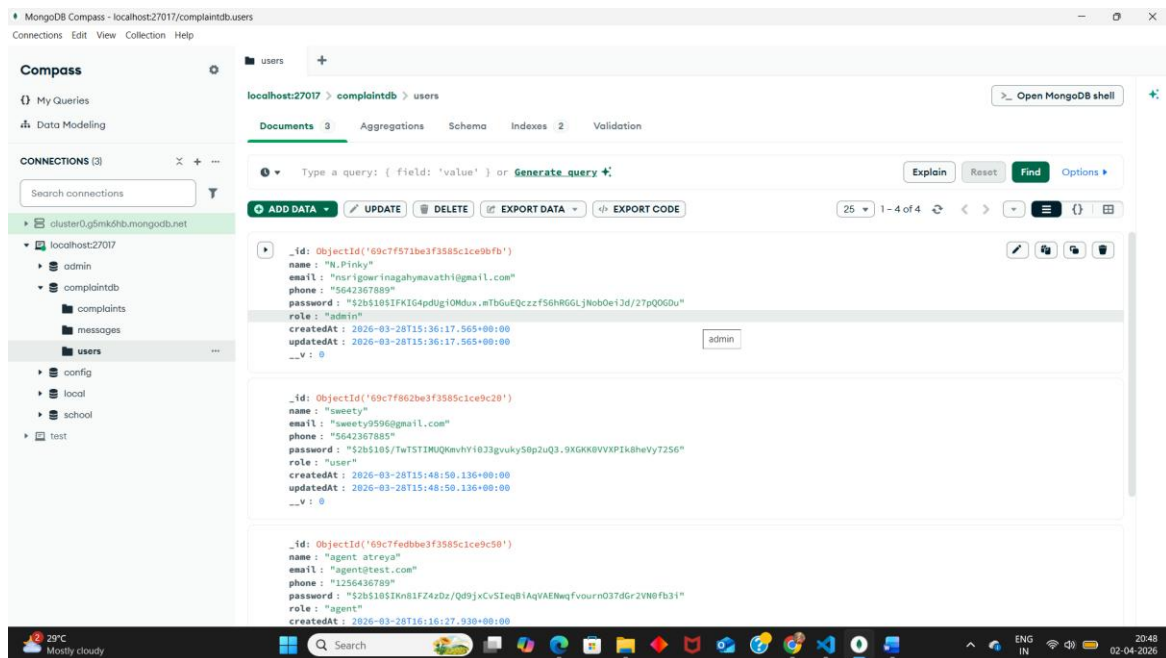
This is agent account and he can see the assigned complaint



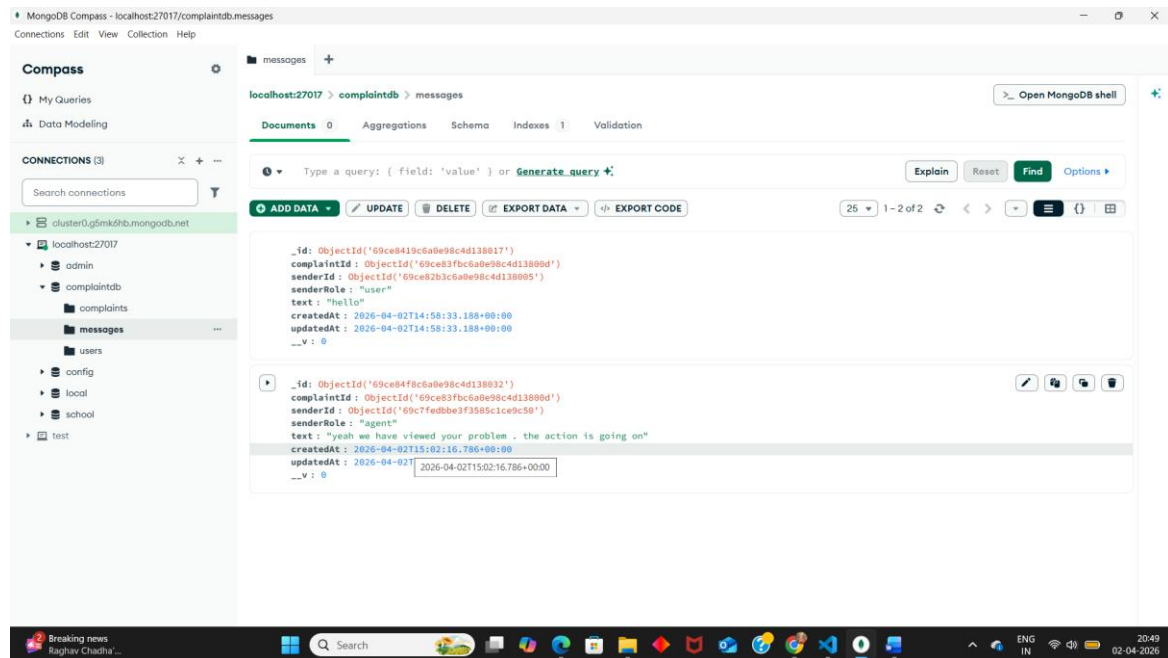
There is chat box so that agent can easily interact with the users



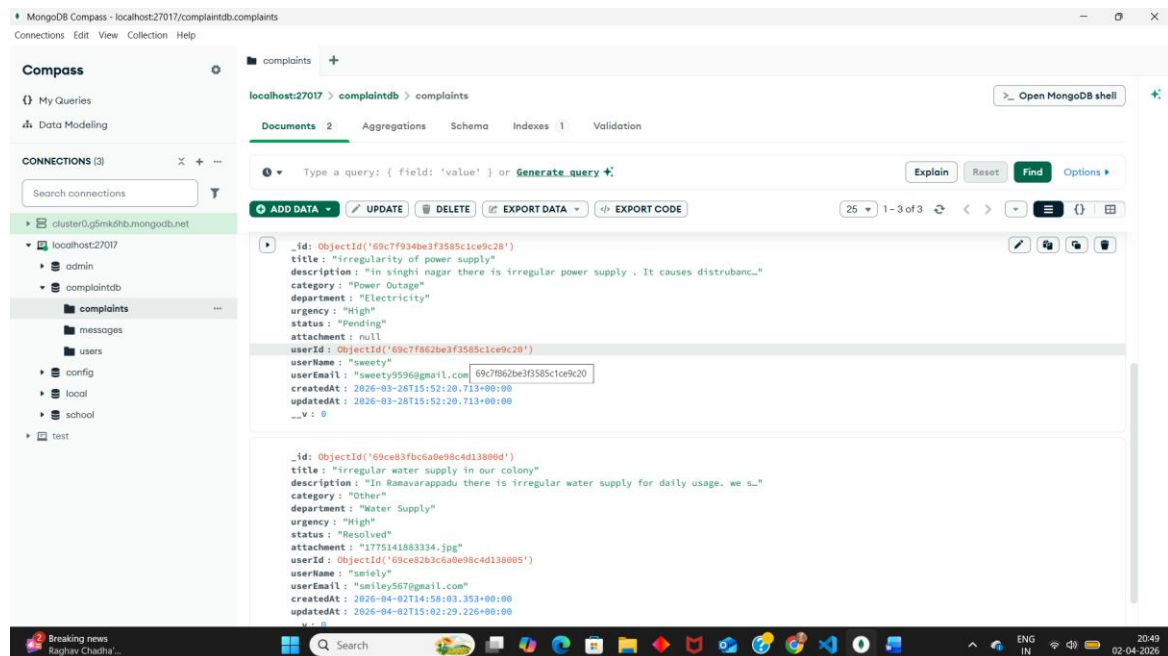
After resolving it will be updated like this



This is mongodb database . This is showing all users



Here it is showing chat between user and agent



The information is stored securely and showing status of the complaints

12. Known Issues

- Real-time chat currently works through REST API polling — Socket.io real-time push notifications can be added in future
- No forgot password feature implemented yet

- Agent accounts must be manually assigned role through MongoDB Compass — no agent registration UI yet
- Image uploads are stored locally in the uploads folder — cloud storage (AWS S3) not yet integrated

13. Future Enhancements

- Add Socket.io for real-time live chat and instant status notifications
- Implement forgot password with OTP verification via email
- Add Google/Facebook social login using OAuth
- Integrate cloud storage (AWS S3 or Cloudinary) for complaint attachments
- Build a mobile app version using React Native
- Add analytics dashboard with charts showing complaint trends by department
- Implement automated email notifications using Nodemailer
- Add complaint priority escalation system for overdue complaints
- Multi-language support for regional users
- Add AI-based complaint category auto-detection